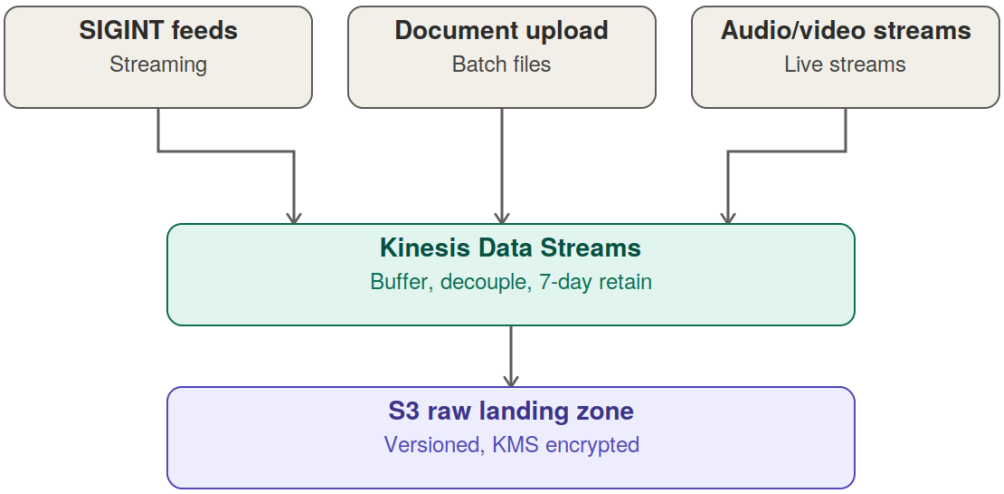
Multi-source intelligence semantic search pipeline

Detailed stage-by-stage architecture breakdown

1. Ingest layer



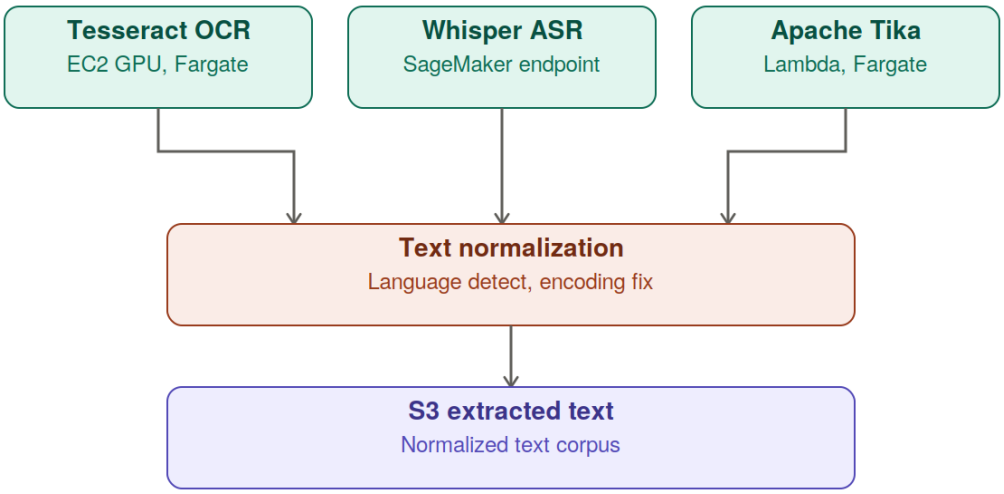
Three collection modalities converge into a single buffered stream before anything is processed:

SIGINT feeds are treated as continuous streaming input, distinct from the batch-oriented document uploads — this matters because streaming SIGINT needs low-latency buffering (Kinesis) rather than a scheduled batch job, since intercept/collection systems typically can't be paused if a downstream stage is slow.

Kinesis Data Streams is the shock absorber: it decouples collection rate from processing rate, so a burst in any one feed doesn't back-pressure the collectors themselves. Seven-day retention gives a real reprocessing window — if the extraction layer has a bug or an outage, you can replay up to a week of ingested data rather than losing it. Auto-scaling shard count is what lets this actually hit 1B objects/day without manual capacity planning.

S3 raw landing zone is the immutable source of truth: versioning ensures nothing is silently overwritten, KMS encryption satisfies at-rest requirements, and a 30-day lifecycle policy keeps this tier from becoming an unbounded cost sink — raw data isn't meant to live here forever, it's a staging area before extraction.

2. Content extraction layer

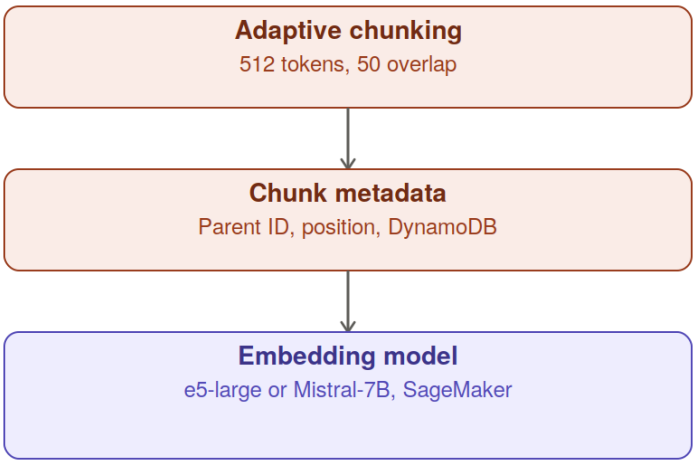


All three tools here are self-hosted open-source rather than managed APIs, which is the load-bearing design decision for this whole layer:

- **Tesseract (OCR)** and **Whisper (ASR)** are both open-source models run on your own compute (EC2/ECS for Tesseract, SageMaker endpoints for Whisper) rather than calling a vendor's OCR/transcription API — this is what makes “no external APIs” achievable, since every other major OCR/ASR service is a hosted API call that would leave the accredited boundary.
- **Apache Tika** handles the more mundane but high-volume case: extracting text from native digital formats (PDF, Office docs, HTML) that don't need OCR or transcription at all. Running it on Lambda/Fargate rather than a persistent GPU fleet reflects that this path is much lighter-weight computationally than OCR or ASR.
- **Text normalization** is where these three heterogeneous outputs converge on one schema — language detection matters a lot here given SIGINT sources are likely multilingual, and encoding fixes address the reality that OCR and legacy document formats routinely produce garbled or mixed-encoding text that would otherwise corrupt downstream embeddings.

One practical note: ASR (Whisper) is typically the most compute-intensive of the three per object — worth confirming its SageMaker endpoint autoscaling is provisioned separately from Tesseract/Tika's compute, since sizing them identically would either starve audio processing or badly over-provision the other two.

3. Chunking and embedding layer

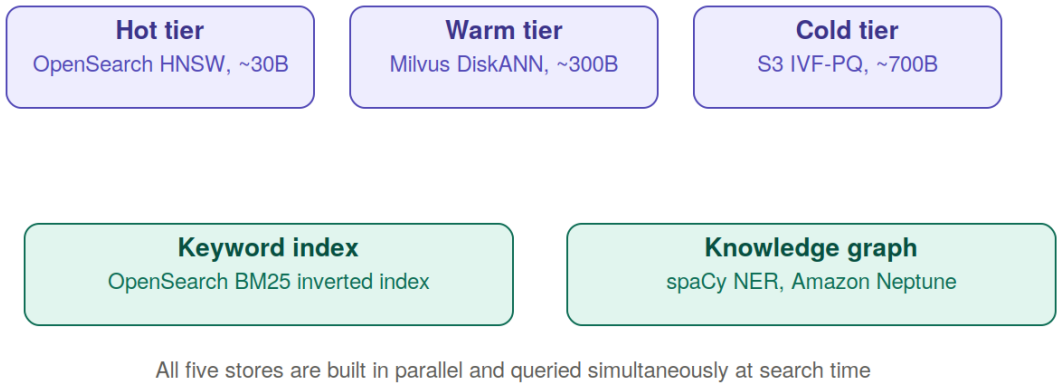


Adaptive chunking at 512 tokens with 50-token overlap, on semantic boundaries rather than fixed character counts: chunking on semantic boundaries (paragraph/sentence breaks) rather than blindly cutting at token 512 avoids severing a sentence mid-thought, which measurably hurts retrieval quality. The 50-token overlap means adjacent chunks share context, so a fact that straddles a chunk boundary is still findable from either side.

Chunk metadata in DynamoDB — parent document ID, chunk position, and timestamp — is what lets you reconstruct where a chunk came from at query time, and lets result assembly later stitch adjacent chunks back into readable context around a hit. DynamoDB's low-latency key lookups are well suited to this since it's queried on nearly every search result.

The embedding model choice (e5-large vs. fine-tuned Mistral-7B) is the single biggest cost/quality lever in this layer. e5-large is smaller, faster, and already strong on multilingual retrieval benchmarks out of the box. A fine-tuned Mistral-7B costs substantially more per token to run at 1B objects/day, and is only worth it if mission vocabulary (codenames, transliterated names, domain jargon) is different enough from general text that a general-purpose model's embeddings cluster poorly — an empirical question worth answering with a retrieval-quality benchmark before committing to the more expensive model at scale.

4. Hybrid vector storage and parallel indexes

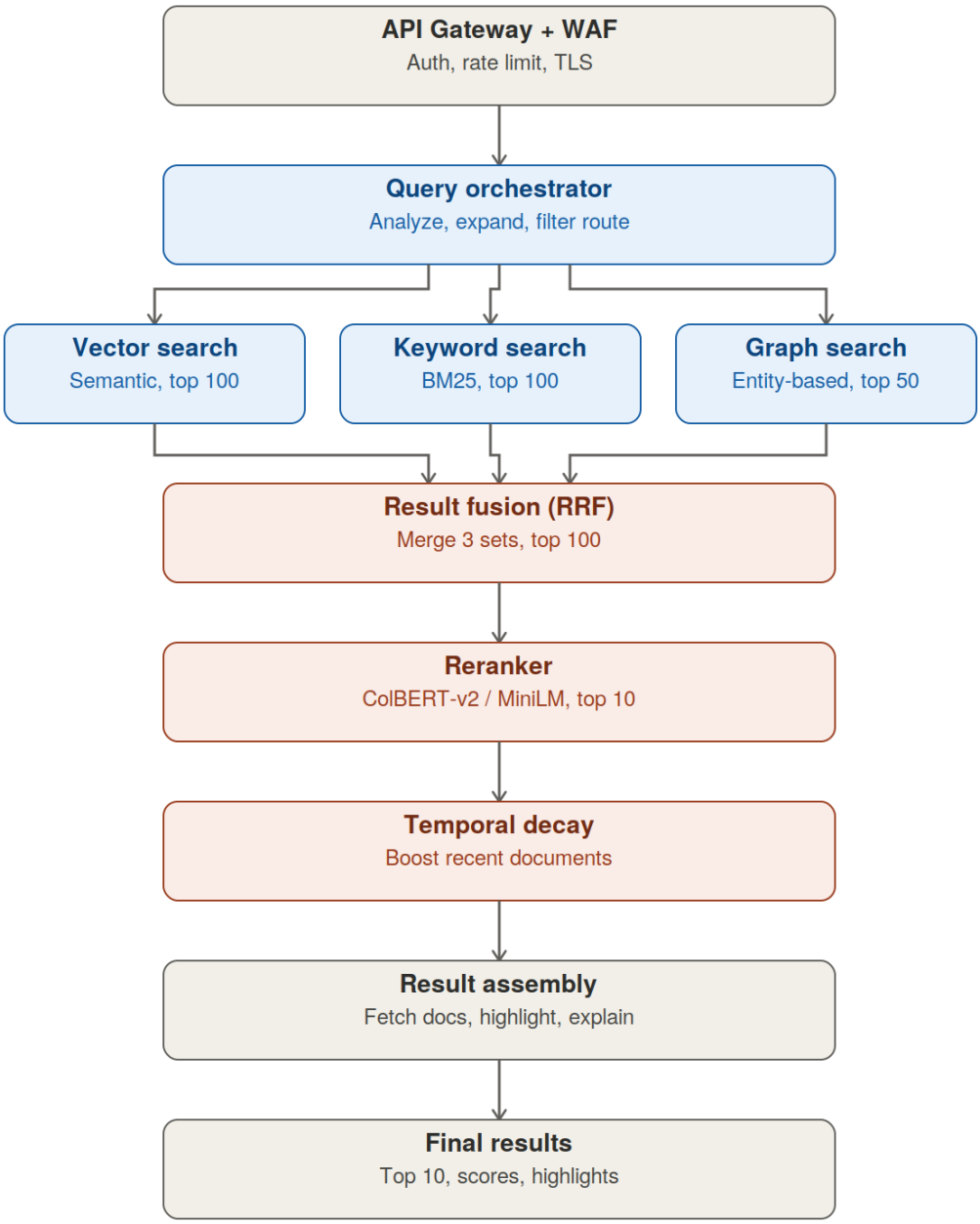


This is the layer where cost engineering matters most, since it's what holds the trillion-object corpus long-term:

- **Hot tier (~30B vectors, OpenSearch HNSW, in-memory)** — the most recently collected 30 days. HNSW in memory gives 97%+ recall and sub-200ms latency, appropriate for data analysts are most likely to be actively searching against right now.
- **Warm tier (~300B vectors, Milvus on EKS, DiskANN)** — DiskANN is specifically engineered for this exact scenario: an index too large for memory but still needing sub-second latency from NVMe SSD rather than falling back to slow disk scans. This is the tier doing the heaviest lifting in terms of raw scale.
- **Cold tier (~700B vectors, S3 + Parquet, IVF-PQ)** — product quantization compresses vectors at the cost of recall (88% here vs. 97%+ hot), the right tradeoff for data searched rarely where a few seconds of latency is acceptable. This is also the cheapest tier per vector by a wide margin.
- **Keyword index and knowledge graph run independently**, not as a bolt-on to the vector index. This means exact phrase matching (BM25) and entity-relationship queries (Neptune/Gremlin) get first-class treatment rather than degraded semantic-only substitutes — important for analyst queries like “show me everything connected to this specific name or facility,” which vector similarity alone handles poorly.

The tradeoff of keeping these five stores separate rather than one unified hybrid index (like Weaviate's built-in hybrid mode) is operational: keeping five systems in sync rather than one or two, in exchange for much finer control over each retrieval mode's behavior.

5. Query pipeline



This is the most component-dense stage, so worth walking through each step in order:

API Gateway + WAF — standard perimeter: authentication, rate limiting, and TLS termination before a query ever reaches application logic.

Query orchestrator does three distinct things worth separating conceptually even though they run in one service: (1) *query analysis* (language detection, intent classification), (2) *query expansion* via a self-hosted Mistral-7B generating 3–5 phrasing variants to improve recall against synonym and phrasing mismatches, and (3) *filter analysis* — estimating filter selectivity to decide routing (a highly selective filter, like an exact date range or classification marking, can route straight to an exact-match path rather than

scanning the full ANN index).

Three parallel searches run simultaneously rather than sequentially: vector (semantic, top 100), keyword (BM25, top 100), and graph (entity-based, top 50). Running them in parallel rather than falling back from one to the next is what keeps end-to-end query latency reasonable despite querying three independent systems.

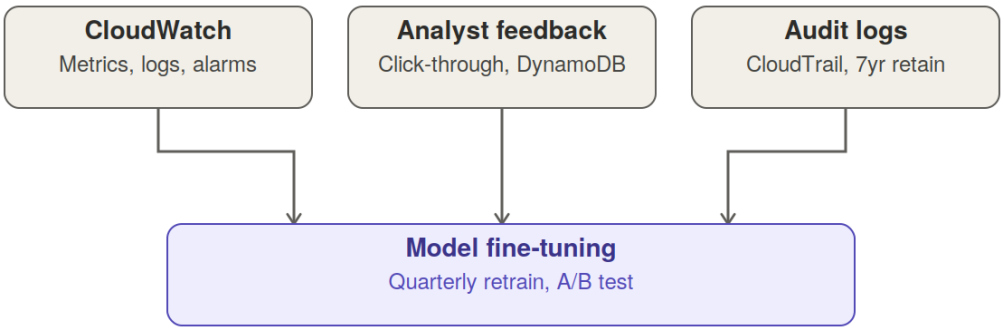
Reciprocal Rank Fusion (RRF) merges these three heterogeneous result sets without needing to calibrate score scales across systems — it works purely on each result's rank within its own list, which sidesteps the hard problem of comparing a BM25 score to a cosine similarity to a graph-traversal score directly.

Reranking (ColBERT-v2 or MiniLM) takes the fused top-100 and does token-level cross-encoder scoring to cut it to the top 10. Cross-encoders are too slow to run over the full corpus, but are exactly the right tool for re-scoring a modest candidate set with much finer relevance judgment than any single first-stage retriever.

Temporal decay scoring boosts recency — appropriate for an intelligence context where a document's operational value often degrades with age, with the decay function and weighting configurable per mission.

Result assembly is the final step that turns ranked chunk IDs into something an analyst can read: pulling full source documents from S3, metadata from DynamoDB, highlighting the matching passage, and generating an explanation of why each result matched.

6. Monitoring and feedback layer



This closes the loop rather than just logging for compliance's sake:

CloudWatch covers operational health — is the pipeline keeping up with 1B objects/day, are any GPU endpoints saturated, are query latencies within SLA.

Analyst feedback (click-through, explicit relevance marks) is the signal that actually improves the system over time — without it, the embedding and reranking models slowly drift out of alignment with how analysts phrase queries and what they consider relevant, especially as mission focus and vocabulary shift.

Audit logs (CloudTrail, 7-year retention) serve a different purpose entirely — this is compliance and accountability infrastructure, not a quality signal. Seven years lines up with typical federal records-retention schedules.

Quarterly fine-tuning with A/B testing is what turns analyst feedback into an actual model update — running it as a scheduled cadence with A/B validation (rather than continuous retraining) is a reasonable way to avoid destabilizing a production system while still keeping it current.

A few architectural principles from the original document cut across every layer above and are worth calling out on their own: **filter-then-rank** (pre-query filtering at the HNSW payload-index and Milvus segment-routing level, so classification/time-based filters narrow the search space before ranking rather than after), and the **C2S/air-gapped/no-external-API** posture that explains why every model here is self-hosted rather than a managed API call.